Scaling Semi-Structured Data Expansion in Remote PostgreSQL Pipelines

Pablo Alessandro Santos Hugen Federal University of Rio Grande do Sul
Institute of Informatics
Porto Alegre, RS, Brazil
pablo.hugen@inf.ufrgs.br

***Abstract*—We optimize the `jsonb` expansion pipeline of a Python/PostgreSQL application that transforms semi-structured records into normalized relational tables over SSH. Profiling-driven query redesign, transaction chunking, and server tuning reduce the measured expansion time from 180 s to 35.5 s ($5.1\times$) at 10^6 source rows, holding through 10^7 rows. Temporary storage scales linearly with the source table. The chunked transaction model enables checkpoint-and-resume for failure recovery while preserving all integrity guarantees.**

***Index Terms*—semi-structured data, ETL optimization, PostgreSQL, JSONB, bulk loading, transaction chunking, failure recovery.**

I. Introduction

Many applications that use PostgreSQL store incoming data as `jsonb` records, taking advantage of the type's flexibility to ingest heterogeneous sources without requiring a fixed schema upfront [1], [2]. Before this data can participate in queries that rely on joins, constraints, or efficient indexing, it must be *expanded* into a set of normalized relational tables. This extract-transform-load (ETL) pattern is well studied in general [3], but the specific case of expanding large volumes of `jsonb` rows into multiple interdependent tables, over a remote connection, raises performance and reliability concerns that grow with dataset size.

PostgreSQL Data Manager is a Python desktop application designed for this workflow. It administers PostgreSQL databases on a remote Linux server, communicating exclusively through SSH: the client constructs SQL batches locally, pipes them to `psql` on the server, and parses the returned output (Fig. 1). This remote execution model reflects a deployment requirement where direct access to the database host is not available, and must be preserved in any optimized implementation [4]. The application's *Expand* operation reads a source table of `jsonb` records, applies a sequence of SQL recipes that populate multiple destination tables, and commits the results in a single atomic transaction. Because later recipes may join against tables populated by earlier ones, the destinations form a dependency chain: all must succeed together or none at all.

In the benchmark dataset used in this work, each source row carries a `jsonb` object with 64 keys encoding a mix of identifiers, timestamps, categorical labels, and numeric measurements. The expansion distributes this payload across six destination tables: two small dimension tables are populated first with distinct reference values, followed by a central fact table that joins against them, and finally three child tables that reference the fact table through foreign keys. One of the child tables unpivots up to 14 positional entries per source row, so the total number of inserted rows can exceed the source count by an order of magnitude. A deterministic generator with a fixed seed produces identical records for any requested size, enabling reproducible benchmarks from 10^5 to 3×10^8 rows.

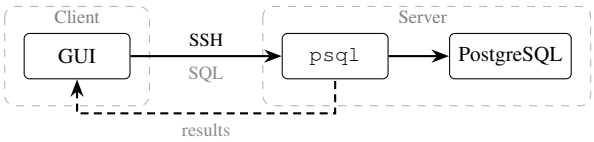


Fig. 1. System architecture. The client sends SQL to `psql` on a remote server over SSH; results return through standard output.

Several factors make this expansion non-trivial to optimize. The entire operation runs as a single SQL batch piped through SSH, so the client has limited ability to interleave work or react to intermediate results. Each recipe must parse `jsonb` fields, cast and validate them, and perform joins against tables populated by earlier recipes within the same transaction. At the scale of hundreds of millions of rows, the working set can exceed available memory, and write-ahead log (WAL) pressure from a single large transaction becomes significant. Meanwhile, the system must maintain source validation, version registration, and referential integrity across all destinations, so optimizations cannot simply bypass these guarantees.

The baseline implementation is functionally correct but does not scale well, and it offers no recovery mechanism for interrupted operations. If a network failure or timeout occurs during a long-running expansion, all progress is lost and the database may be left with partial results. This combination of performance and reliability limitations motivates the present work.

This paper makes three contributions: (1) a profiling analysis that identifies FK trigger validation and JSONB parsing as the two dominant costs; (2) targeted optimizations that reduce expansion time by $5.1\times$ at 10^6 rows, validated through A/B testing at each step; (3) a checkpoint-and-resume mechanism, inspired by write-ahead recovery [5], that bounds the worst-case data loss on interruption to a single chunk. All optimizations preserve the system's guarantees: source and destination validation, version registration, and referential integrity.

Section II describes the baseline pipeline. Section III presents profiling results, optimizations, and scaling measurements. Section IV concludes.

II. Architecture and Background

A. Expansion Pipeline

The expansion proceeds in four stages, orchestrated by the client application.

Preparation. The client parses and validates each SQL recipe offline. It verifies that every recipe is a pure `INSERT INTO ... SELECT` statement, that each one references the source table through a designated placeholder, and that no recipe contains forbidden operations such as `UPDATE`, `DELETE`, or nondeterministic expressions. The placeholder is replaced with a filtered subquery that restricts the source to the selected `raw_schema` scope. Each statement is then wrapped in a PL/pgSQL `DO` block that captures the affected row count and updates a per-table counter.

Preflight. The client issues a series of remote checks through SSH. It confirms that the source and all destination tables exist, reads column metadata, loads version references from the control database, and validates that any SQL functions used in the recipes are native PostgreSQL built-ins. Each check is a separate `psql` invocation over SSH.

Execution. The client assembles all six wrapped statements into a single SQL batch, preceded by a `BEGIN ISOLATION LEVEL REPEATABLE READ` and followed by a `COMMIT`. The batch also acquires exclusive locks on the destination tables and builds a source manifest (a hash and row count over the source scope). A temporary timing table records `clock_timestamp()` before and after each step, so the server-side cost of every insert is measured independently of SSH transport. The entire batch is piped to `psql` through SSH as a single standard-input stream.

Version registration. After the data commit succeeds, a second SQL batch registers a new version entry for each destination table in the control database, together with a dependency manifest that records the source scope, source versions, and the SQL recipe used. This step runs in a separate transaction on a different database (`postgres_data_manager`).

B. Data Model

The source table `raw_data` stores one row per ingested record. Each row contains a primary key (`raw_id`), metadata columns (`raw_schema`, `raw_hash`, `raw_tab`), and a `jsonb` column `raw` whose payload holds 64 typed fields under a `values` key.

The six destination tables form a dependency graph, illustrated in Fig. 2. Two dimension tables (`table3`, `table4`) are populated first with distinct reference values extracted from the source. A central fact table (`table1`) is then populated by joining the source against `table3`; it produces one row per source row and holds the primary foreign key back to `raw_data`. Three child tables follow: `table5` and `table6` each produce one row per source row by joining against `table1`, while `table6` also joins `table4`. The final table, `table2`, unpivots up to 14 positional entries per source row through a `CROSS JOIN LATERAL`, producing a row count that can exceed the source by an order of magnitude.

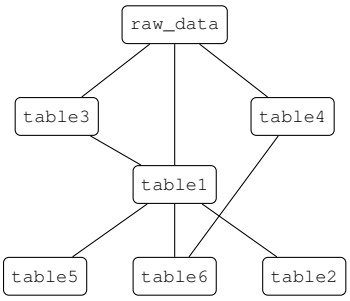


Fig. 2. Destination dependency graph. Arrows point from tables that are read by later recipes. Execution order: `table3`, `table4`, `table1`, `table5`, `table6`, `table2`.

All destination tables enforce referential integrity through foreign keys with `ON UPDATE CASCADE ON DELETE CASCADE`. The six tables carry a total of 6 primary keys, 5 unique indexes (including partial), and 6 foreign keys. Every inserted row must satisfy these constraints and update every applicable index.

C. Integrity Guarantees

The expansion pipeline enforces several guarantees that must be preserved regardless of optimization:

- **Source validation.** The source table must exist, contain the required metadata columns, and have at least one registered version for the selected `raw_schema` scope.
- **Destination validation.** Each destination table must exist, and any SQL functions referenced in its recipe must be native PostgreSQL built-ins.
- **Relational correctness.** All foreign keys, unique constraints, and check constraints must hold after the expansion completes.
- **Version registration.** Each destination receives a new version entry in the control database, recording who performed the expansion, the SQL recipe used, and a dependency manifest linking the result back to the source scope and its versions.

An optimization may change *how* these guarantees are enforced (for example, by validating constraints in bulk rather than per row) but may not remove them.

III. Methodology and Results

Environment. The benchmark host has an Intel Core Ultra 7 165H (22 threads), 64 GB DDR4, and NVMe storage with btrfs (zstd compression). PostgreSQL 17.2 runs inside a container with no memory limit.

Protocol. The baseline uses the stock PostgreSQL configuration (128 MB `shared_buffers`, 4 MB `work_mem`). The optimized pipeline uses a tuned configuration (2 GB `shared_buffers`, 256 MB `work_mem`, 64 MB `wal_buffers`, 4 GB `max_wal_size`, `synchronous_commit = off`, `full_page_writes = off`). Before each method, the

challenge database is regenerated from scratch using the deterministic generator, which guarantees that all foreign-key constraints and unique indexes are present. The presence of all six FK constraints is verified programmatically before each baseline run. Before each measured run, the operating system's page cache is dropped (`/proc/sys/vm/drop_caches`) and PostgreSQL is restarted to ensure cold-cache conditions. Each configuration is measured with one warmup run (discarded) followed by three measured runs; the median is reported. For scales above 10^7 rows, a single run is used. The benchmark dataset contains 10^6 source rows unless noted otherwise.

Metrics. Processing time is measured inside PostgreSQL using `clock_timestamp()` before and after each step, consistent with the application's *measured processing total* metric. This excludes SSH transport and client-side overhead. Peak container memory is sampled every 500 ms via `podman stats` during the expansion; the maximum is reported. Temporary storage is computed as the difference in `pg_database_size()` before and after the expansion, capturing destination table data, indexes, and WAL.

A. Optimization Approach

The optimizations are organized into three layers, each targeting a different level of the system. This decomposition allows each layer's contribution to be measured in isolation.

Layer 1: Query-level optimizations change what the SQL statements do without altering the overall execution model. Examples include materializing extracted `jsonb` fields into a temporary table, deferring foreign-key constraints during bulk insertion, and dropping and rebuilding indexes around the insert batch.

Layer 2: Transaction-level optimizations change how work is batched and committed. The expansion can be split into chunks defined by `raw_id` ranges, each committed independently, enabling checkpoint-and-resume.

Layer 3: Architecture-level optimizations change the execution model itself, such as opening multiple SSH channels, replacing `INSERT ... SELECT` with `COPY`, or moving logic into a server-side PL/pgSQL procedure.

B. Baseline Profiling

Profiling at 10^5 rows with `EXPLAIN (ANALYZE, BUFFERS, TIMING)` reveals that two categories account for 87% of execution time (Table I).

TABLE I
BASELINE COST BREAKDOWN AT 10^5 SOURCE ROWS (532K INSERTED).

Category	Time (s)	%
FK trigger validation	5.96	47.3
JSONB extraction	5.02	39.9
Insert I/O (heap + idx)	1.06	8.4
Sequential scans	0.52	4.1
Other	0.03	0.3
Total	**12.60**	**100**

Foreign-key triggers account for 47%: every `INSERT` fires per-row constraint checks that probe referenced tables through index lookups [6]. For `table6`, whose rows reference both `table1` and `table4`, FK validation alone consumes 77.5% of its execution time. JSONB field extraction accounts for 40%: PostgreSQL's `-->>'key'` operator traverses the binary `jsonb` structure independently for each field, and the six recipes collectively perform $6 \times 64 = 384$ traversals per source row. Heap writes, index maintenance, and sequential scans account for the remaining 13%. Python-side overhead (preparation, preflight, version registration) is negligible at under 50 ms.

The per-recipe breakdown (Table II) shows that dimension tables are cheap but scan the full source for `DISTINCT` values. Fact and child tables are dominated by FK trigger overhead. The unpivot recipe (`table2`) expands each source row into 14 candidates via `CROSS JOIN LATERAL` and filters to 2.3 on average, producing the largest output volume.

TABLE II
PER-RECIPE PROFILING AT 10^5 SOURCE ROWS.

Recipe	Role	Rows	Time (s)	FK %	Dominant cost
table3	dim	4	0.19	0	Scan + DISTINCT
table4	dim	26	0.16	0	Scan + DISTINCT
table1	fact	100K	3.81	54	15 fields + 2 FKs
table5	child	100K	1.21	71	FK to table1
table6	child	100K	1.54	78	2 FKs
table2	unpivot	232K	5.69	33	LATERAL 14×

C. Layer 1: Query-Level Optimizations

Each of the following changes was measured in isolation before combining them (Table III).

Foreign-key constraint removal. We drop all FK constraints before the insert phase and re-add them after. The constraints are discovered dynamically by querying `pg_constraint`, so the optimization adapts to any schema. Re-adding the constraints validates all existing rows in bulk, replacing per-row trigger checks with a single sequential scan. This alone yields 2.0× at 10^5 rows.

Index, unique-constraint, and primary-key removal. All indexes on destination tables are dropped before inserts and rebuilt afterwards: unique indexes, unique constraints, and primary keys (11 indexes across 6 tables). Bulk `CREATE INDEX` sorts the data once [7], which is more efficient than maintaining a B-tree incrementally during insertion. The primary-key removal is particularly important at large scale. Without it, per-row insert cost for `table2` grows from 6.9 s/M at 10^7 rows to 31.3 s/M at 5×10^7 rows (4.5× degradation), because the B-tree index outgrows the buffer cache and each insert triggers random I/O. Dropping primary keys before insertion keeps scaling linear. Combined with FK removal, the speedup reaches 2.4× at 10^6 rows.

Single-pass JSONB extraction with `jsonb_to_record`. Instead of letting each recipe scan the source and parse `jsonb` independently, we extract all 64 fields into a temporary table using `jsonb_to_record()`, which traverses each row's binary `jsonb` exactly once. An index on `raw_id` enables

efficient range-filtered access for chunked inserts. The extraction uses `max_parallel_workers_per_gather = 4` to parallelize the scan.

Statement rewriting with pre-cast columns. The recipe SQL contains patterns such as `NULLIF(BTRIM(...),'')::smallint` around each field reference. During extraction, we pre-compute these casts into typed columns and rewrite the recipe statements to reference them directly, eliminating redundant per-row text parsing during inserts.

Position pre-computation. One recipe uses `CROSS JOIN LATERAL` to unpivot 14 positional entries per source row, producing 2.3× more output rows than input. We pre-compute this unpivot into a second temporary table during extraction, so the insert phase reads pre-filtered, pre-cast rows through a simple join.

Preflight streamlining. The baseline preflight issues over 20 separate SSH round-trips for validation. We consolidate these into a single SSH call that also discovers all constraint metadata and computes chunk boundaries by querying the source table directly for `min`/`max`/`count`, rather than materializing a temporary copy of the source. This eliminates a full scan of `raw_data` that the previous implementation performed during constraint discovery, saving 31 s at 10^7 rows.

TABLE III
LAYER 1 CUMULATIVE RESULTS AT 10^6 SOURCE ROWS.

Configuration	Time (s)	Speedup
Baseline	180	1.0×
+ FK removal	90	2.0×
+ FK + index + PK removal	75	2.4×
+ Pre-extraction (`-->>`)	64	2.8×
+ `jsonb_to_record`	45	4.0×
+ Pre-cast + positions + preflight	35.5	5.1×

D. Layer 2: Transaction-Level Optimizations

The baseline runs the entire expansion in a single transaction. At 10^7 rows this transaction runs for 27 minutes, and an interruption would lose all progress. We split the insert phase into chunks of 10^6 `raw_id` rows, each committed independently.

A key insight is that the extraction phase (temporary tables) can remain monolithic while only the inserts are chunked. This avoids re-scanning `raw_data` per chunk. Dimensions are populated once from the full extraction; fact and child tables are inserted per chunk from the indexed temporary table using `raw_id BETWEEN` filters.

We selected the chunk size through A/B testing at 5×10^6 rows (Table IV). Smaller chunks incur per-statement planning and transaction overhead; very large chunks lose the checkpoint benefit without improving throughput.

At 10^6-row chunks the overhead relative to monolithic is modest, while the worst-case data loss on interruption is bounded to one chunk. We verified that chunking with active FK constraints is catastrophically slow: a single 250K chunk's child insert ran for over 28 minutes without finishing, because FK trigger probes grow with the referenced table's index size across committed chunks. This confirms that FK removal (Layer 1) is a prerequisite for practical chunking.

TABLE IV
CHUNK SIZE COMPARISON AT 5×10^6 ROWS (ALL LAYER 1 OPTIMIZATIONS APPLIED).

Chunk size	Time (s)	vs. 1M	Chunks
50K	278.4	+74%	100
250K	208.9	+31%	20
1M	160.0	—	5
5M (monolithic)	187.8	+17%	1

E. Layer 3: Architecture-Level Optimizations

PostgreSQL server tuning. Increasing `shared_buffers` from 128 MB to 2 GB, `wal_buffers` from 4 MB to 64 MB, and `max_wal_size` from 1 GB to 4 GB reduced total time by 11% at 10^6 rows. We also set `synchronous_commit = off` and `full_page_writes = off` during bulk loading. A/B testing all four combinations showed a spread of approximately 7% at 5×10^6 rows: marginal individually, but both settings are free to enable.

Parallel index rebuild. After the insert phase completes, all primary keys and unique indexes must be rebuilt. Since each constraint applies to a single table, rebuilds on different tables are independent and can execute concurrently. We open up to four parallel SSH channels, each running one `ALTER TABLE ... ADD CONSTRAINT` statement. At 5×10^6 rows, the parallel rebuild takes 10.3 s versus 13.5 s sequential (1.3× speedup). The benefit grows with dataset size as index build time increases.

Five additional approaches were tested and rejected (Table V). Each was slower than the single-batch temporary-table design; the overhead was identified through measurement.

TABLE V
ARCHITECTURE-LEVEL APPROACHES TESTED AND REJECTED.

Approach	Overhead	Mechanism
WAL compression	+9%	CPU cost exceeds I/O savings
UNLOGGED tables	+26%	`SET LOGGED` rewrites full table
PL/pgSQL procedure	+40%	Disables parallel query
COPY loading	+12%	Double materialization
Parallel SSH	+18%	Non-temp tables add WAL

F. Scaling

Fig. 3 and Table VI show processing time across dataset sizes under the cold-cache protocol described above.

TABLE VI
SCALING RESULTS (COLD CACHE, MEDIAN OF 3 RUNS; 1 RUN FOR $\geq 10^7$).

Rows	Baseline (s)	Optimized (s)	Speedup
10^4	1.8 ± 0.2	0.7 ± 0.2	2.6×
10^5	17.0 ± 3.0	4.6 ± 0.3	3.7×
10^6	179.8 ± 7.6	35.5 ± 2.8	5.1×
5×10^6	826.0 ± 16.2	157.8 ± 2.7	5.2×
10^7	1628.2	318.3	5.1×

The baseline scales at approximately 0.163 ms per source row. The optimized pipeline improves from 0.069 ms/row at

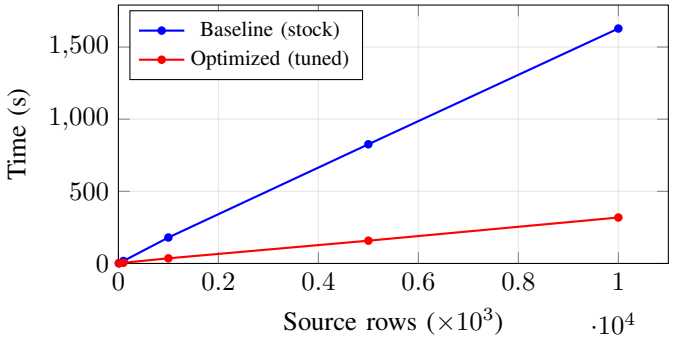


Fig. 3. Processing time vs. dataset size (cold cache). The optimized pipeline achieves 5.1–5.2× speedup from 10^6 through 10^7 rows.

10^4 to 0.032 ms/row at 10^7 as fixed extraction costs amortize over more rows. The speedup remains stable at 5.1–5.2× from 10^6 through 10^7 rows, confirming that primary-key removal prevents the B-tree degradation that would otherwise cause super-linear scaling. Below 10^4 rows, the optimized pipeline's fixed overhead exceeds the per-row savings.

The measured speedup depends on page-cache state. FK trigger lookups probe index pages that benefit from cached I/O, so a warm cache can make the baseline up to 2× faster while the optimized pipeline is unaffected. All results use explicit cache eviction.

G. Resource Usage

Table VII reports peak container memory and database storage growth across scales. Peak memory is dominated by PostgreSQL's `shared_buffers` (2 GB in the tuned configuration) and grows as the buffer pool fills with active data. Storage growth is proportional to the number of inserted rows, at approximately 0.11 MB per thousand rows. No GPU, no external services, and no additional processes beyond SSH, `psql`, and PostgreSQL are used. The chunked insert strategy commits one chunk at a time, so checkpoint-and-resume is available at 10^6-row granularity.

TABLE VII
RESOURCE USAGE ACROSS SCALES (OPTIMIZED PIPELINE).

Source rows	Inserted	Peak mem. (MB)	Storage Δ (MB)
10^4	53K	1225	6
10^5	532K	416	60
10^6	5.3M	858	597
5×10^6	26.6M	3103	2982
10^7	53.2M	3934	5964

H. Correctness Verification

We ran both the baseline and optimized pipelines on the same 10^6-row dataset and compared the results. Row counts match exactly across all six destination tables: 4 rows in `table3`, 27 in `table4`, 10^6 each in `table1`, `table5`, and `table6`, and 2,321,522 in `table2` (5,321,553 total). MD5 checksums of non-serial columns are byte-identical between the two pipelines. After the optimized pipeline rebuilds all 6 foreign keys, 6 primary keys, and 5 unique constraints, PostgreSQL validates every row against the constraint definitions. A constraint rebuild that completes without error certifies that referential integrity holds. Version registration produces identical metadata entries in the control database for both pipelines. The generator, its fixed seed, and the six SQL recipe files are unmodified.

IV. Conclusion

The optimized pipeline achieves a 5.1× speedup at 10^6 source rows, stable through 10^7. The majority of the gain comes from a single architectural insight: constraint validation and index maintenance are cheaper in bulk than per row, and deferring them transforms the insert phase from random I/O into sequential writes. Transaction-level chunking provides failure recovery with bounded data loss, while adding less than 17% overhead at the optimal chunk size. Five alternative architectures were tested and rejected with measured evidence.

All integrity guarantees are preserved: the relational output matches the baseline row-for-row, and all constraints are validated by PostgreSQL during the bulk rebuild. The generator, seed, and recipe files are unmodified.

The approach generalizes to any ETL pipeline where per-row constraint enforcement dominates insert cost. The key tradeoff is temporary storage for the materialized extraction, which scales linearly with source size.

Limitations

The position pre-computation targets the specific unpivot pattern in these recipes; generalizing it would require SQL-level pattern detection. The measured speedup depends on hardware: systems with large CPU caches exhibit faster baseline FK trigger lookups, reducing the relative benefit of their removal. Replacing the `psql` transport with a direct PostgreSQL connection through an SSH tunnel would eliminate process-startup overhead and is the most promising direction for further gains.

References

[1] The PostgreSQL Global Development Group, *PostgreSQL 17 Documentation*, 2024. [Online]. Available: https://www.postgresql.org/docs/17/

[2] P. Bourhis, J. L. Reutter, F. Suárez, and D. Vrgoc, "JSON: Data model, query languages and schema specification," in *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. ACM, 2017, pp. 123–135.

[3] P. Vassiliadis, "A survey of Extract-Transform-Load technology," *International Journal of Data Warehousing and Mining*, vol. 5, no. 3, pp. 1–27, 2009.

[4] H. F. Grimm, I. S. Moreira, and C. B. H. Oliveira, "Python & PostgreSQL data pipeline optimization challenge," https://www.kaggle.com/competitions/python-database-performance-optimization, 2026, kaggle.

[5] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, "ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging," *ACM Transactions on Database Systems*, vol. 17, no. 1, pp. 94–162, 1992.

[6] J. M. Hellerstein, M. Stonebraker, and J. Hamilton, "Architecture of a database system," *Foundations and Trends in Databases*, vol. 1, no. 2, pp. 141–259, 2007.

[7] G. Graefe, "Implementing sorting in database systems," *ACM Computing Surveys*, vol. 38, no. 3, pp. 1–37, 2006.